ITNPAI1 PROJECT REPORT

Tree-Car Classifier

Sonal Edwin
3037530

INTRODUCTION

Image classification is a computer vision task which involves categorizing images into predefine classes. Some of its applications include visual search, object recognition, medical imaging etc.
In this project, Images of cars and trees are categorized into 2 classes.
The data represents 2 places Stirling (Scotland) and Kerala (India) with 200 pictures from each place.

	Stirling	Kerala
Cars	100 (Google street)	100 (Google street)
Trees	100 (Camera)	100 (Camera)

A classification model is built using this data. The images from each place is split into train (80), validation (20) and test (100) sets.

DESCRIPTION OF PROPOSED SOLUTION

The data after being loaded was augmented by rescaling, rotating, shifting height and width with a range of 20%, zooming and flipping the training data.

A Sequential model was used in this classification task with the help of tensorflow which the most widely used deep learning framework and has power and flexible API's such as Keras, that make it easy to define, train, and evaluate deep learning models. Since our task is simple, with a single input and a single output where the flow of data moves in a sequential manner, this model was considered.

The model architecture consists of convolutional layers with 32 filters, each of size 3x3 pixels. These layers use the 'relu' activation function to convert negative values to 0 and keep positive values unchanged. The input shape is set to 224x224 pixels with 3 channels.

After the convolutional layers, a max pooling layer with a pooling region of 2x2 is applied, taking the maximum value of the relu activations.

Next, the data is flattened into a one-dimensional vector before being passed to a dense layer with 224 neurons and the relu activation function.

The final layer is a single dense layer, which produces a single output. The output represents either 0 or 1, as it is passed through a sigmoid activation function.

NOTE: I chose 224 neurons for the dense layer to match up with the input shape. In order to prevent overfitting, I did try with other values (64) but did not see any significant change in the result.

The model is then compiled using the 'adam 'optimizer and the loss is specified using binary cross entropy because this is a binary classification problem. Finally, the metrics we track in this case is the accuracy.

RESULTS & DISCUSSION

The model was trained using the training set and evaluated using the validation set to see how well the model is performing in real time simultaneously.
It has a batch size of 32 with a total of 3 batches and has been trained with 10 Epochs.

Data	Stirling	Kerala	Kerala (Train) – Stirling (Test)	Stirling (Train)- Kerala (Test)
Accuracy	0.94	0.97	0.77	0.97
Precision	0.89	0.94	0.69	0.94
Recall	1.00	1.00	0.98	1.00

The actual results:

```
For the Stirling Model:
Precision: 0.89285713 Recall: 1.0 Accuracy: 0.94
```

```
For the Kerala Model:
Precision: 0.9433962 Recall: 1.0 Accuracy: 0.97
```

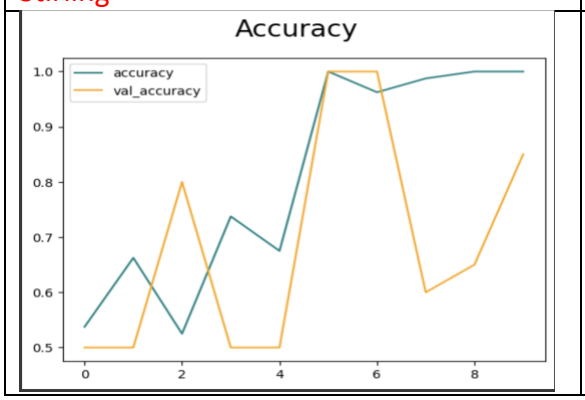
```
For the Model trained using Kerala's Train set and Stirling's validation set
The results on the Stirling test set is:
Precision: 0.6944444 Recall: 0.98039216 Accuracy: 0.77
```

```
For the Model trained using Stilring's Train set and kerala's validation set
The results on the Kerala test set is:
Precision: 0.9433962 Recall: 1.0 Accuracy: 0.97
```

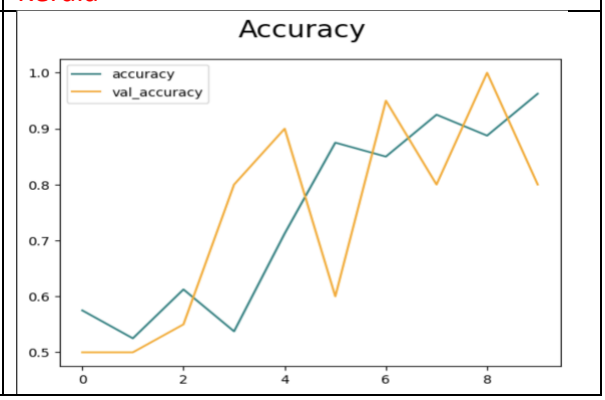
The model trained on Kerala had higher accuracy to the model trained on Stirling data.
By crossing the datasets, the model trained with Stirling as train data and Kerala as validation data had given 0.97 accuracy when tested on Kerala test set.
The model trained with Kerala as train data and Stirling as validation data had given 0.77 accuracy when tested on Stirling test set.

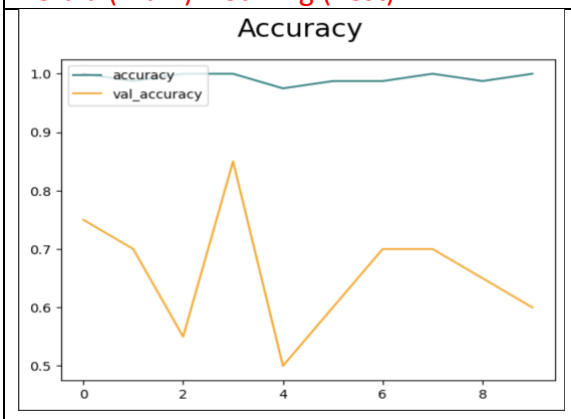
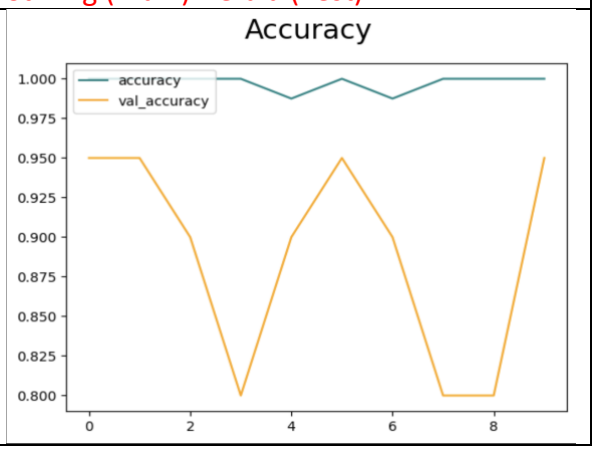
Plot of Train Accuracy and Validation Accuracy for models trained in the respective location

Stirling



Kerala



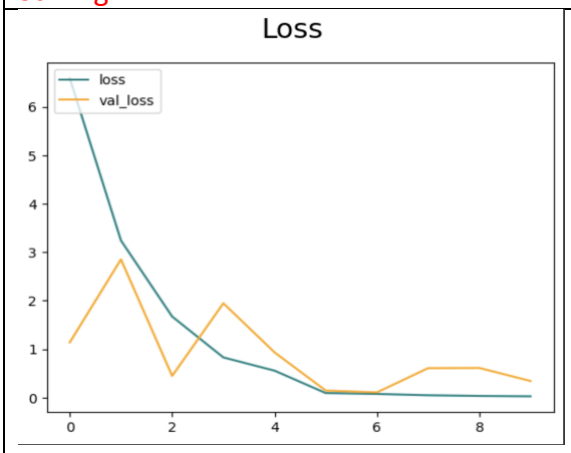
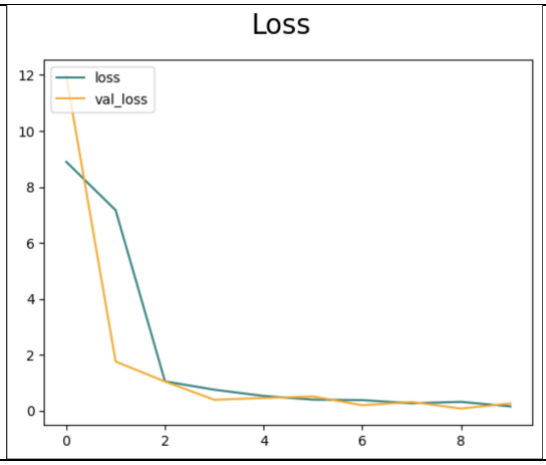
Kerala (Train) – Stirling (Test)	Stirling (Train)-Kerala (Test)
	

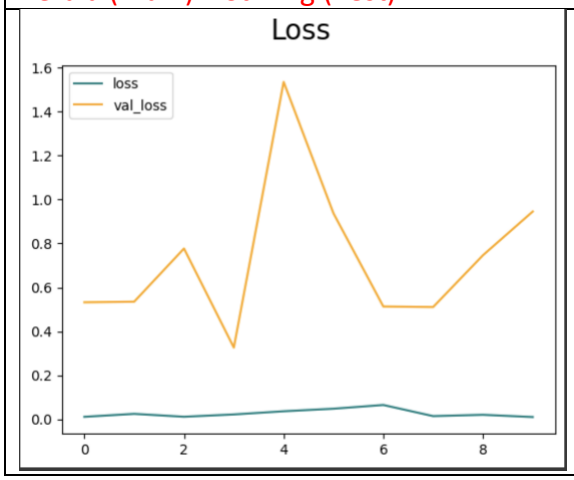
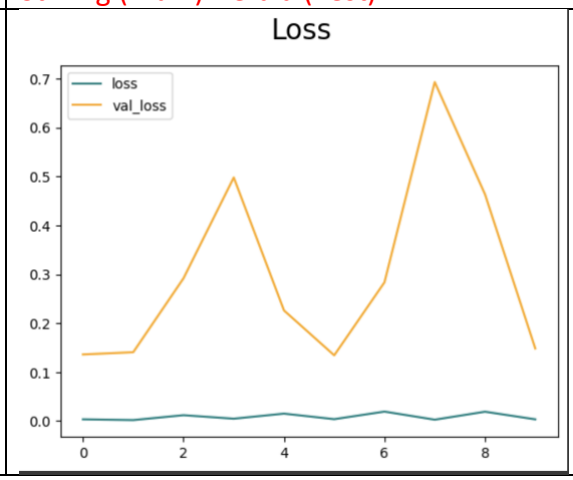
From the above plots we can see that, the model trained on Stirling dataset has been huge jump of the training accuracy in the 4th Epoch and has stayed there at 90 and 100% for the remaining Epochs.
The validation accuracy jumps similarly at the 4th Epoch but crashes at the 7th Epoch and slowly reaches 90% by the end of the training.

The Kerala model, the training accuracy has a jump at the 3rd Epoch and then followed by an almost gradual increase till the last Epoch.
It's validation accuracy similar jump at the 4th Epoch but immediately crashes after which the accuracy increases and decreases mildly.

In the models trained with the crossing of data, the training accuracy always remained at the top within the 85 – 100 range and the validation accuracy was poor with rise and slumps until one gradually decreased to 60% and the other one increasing to 95%.

Plot of Train Accuracy and Validation Accuracy for the respective locations

Stirling	Kerala
	

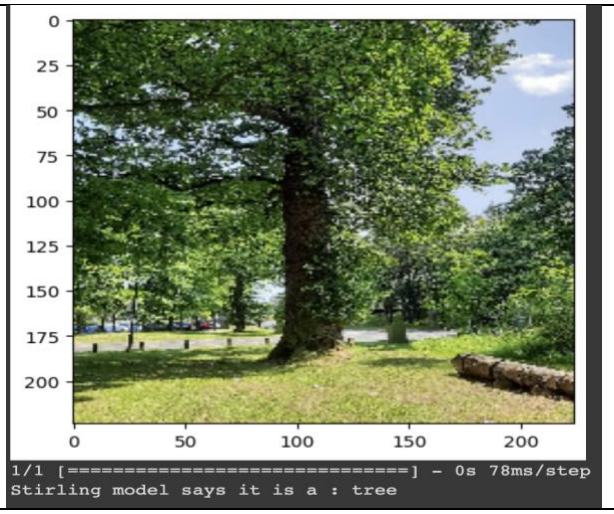
Kerala (Train) – Stirling (Test)	Stirling (Train)-Kerala (Test)
	

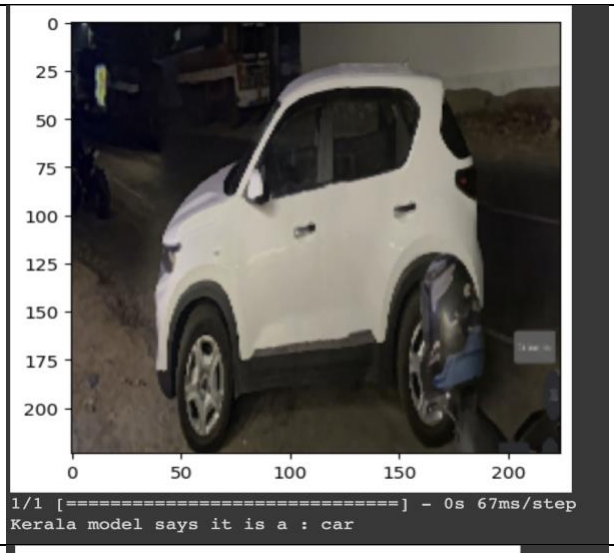
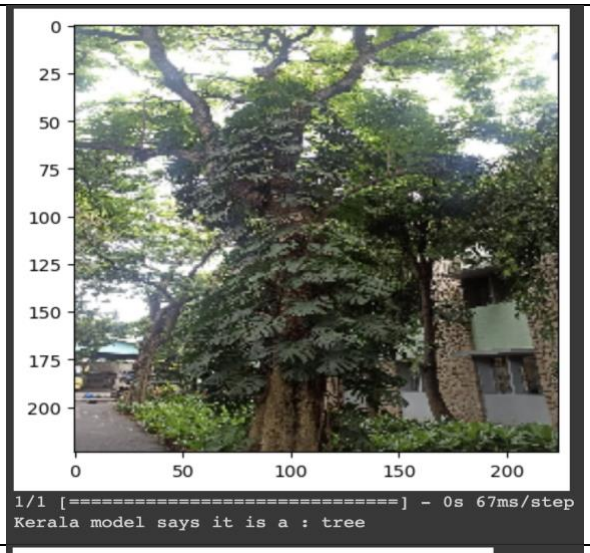
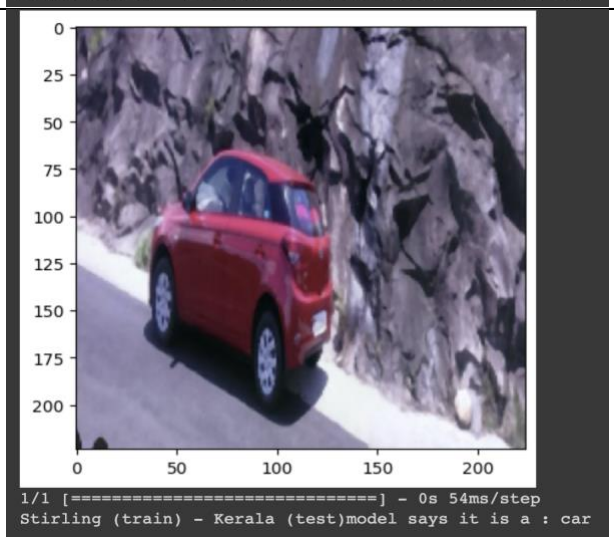
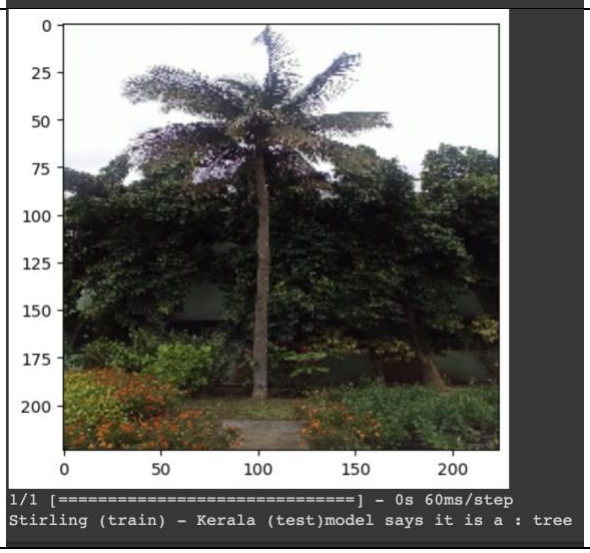
Training loss for the Kerala and Stirling model gradually decreases to almost 0, which is also same for the validation loss of the Kerala model. Whereas there are few jumps followed by an almost 0 loss for the Stirling dataset.

Crossing data models had a stagnant 0 -20 % training loss and had high increase and decrease of validation losses which was almost up to 90% in one and 10% in the other.

PREDICTIONS

Over 20 Images were predicted for each model

Stirling		

Kerala		
Kerala (Train) – Stirling (Test)		
Stirling (Train)- Kerala (Test)		

All models gave out 100% accurate results except the Kerala(Train) Stirling(Test) model that gave a few wrong predictions.

Example:

CONCLUSION:

This project enabled understanding of various techniques to build, train and run an Image classification task. From the data collection to its augmentation and predicting these Images after training was interesting. The model was able to correctly distinguish between a Car and Tree which satisfied the aim of the project.

REFERENCES:

[1] N. Sharma and V. Jain, An Analysis Of Convolutional Neural Networks For Image Classification, https://www.sciencedirect.com/science/article/pii/S1877050918309335

[2] N. Renotte, Build a Deep CNN Image Classifier with ANY Images, https://www.youtube.com/watch?v=jztwpsIzEGc&t=1063s

[3] N. Renotte, ImageClassification /Getting Started.ipynb, https://github.com/nicknochnack/ImageClassification/blob/main/Getting%20Started.ipynb

[4] J. Patel, Image-Classification-CNN-Keras /Solution - CNN Image Classification.ipynb, https://github.com/Coding-Lane/Image-Classification-CNN-Keras/blob/main/Solution%20-%20CNN%20Image%20Classification.ipynb

[5] Coding Lane, Image Classification using CNN Keras | Full implementation, https://www.youtube.com/watch?v=J1jhfAw5Uvo

[6] C. Patnaik, Deep_Learning_Bootcamp/Assignment_2 /Assignment2_Help_Notebook.ipynb, https://github.com/dphi-official/Deep_Learning_Bootcamp/blob/master/Assignment_2/Assignment2_Help_Notebook.ipynb

[7] R. Dasari, Walkthrough on how to fetch, load, and pre-process image dataset for Deep Learning by Raviraj Dasari, https://www.youtube.com/watch?v=OOX42WVobZo

ACKNOWLEDGEMENT:

Thanks to Carol C Morris and Suhaas Edwin for taking and sending the Tree pictures for the Kerala dataset.